

[Introduction](#) > Basics of Prompting[Copy page](#) ▾

Basics of Prompting

Prompting an LLM

You can achieve a lot with simple prompts, but the quality of results depends on how much information you provide it and how well-crafted the prompt is. A prompt can contain information like the *instruction* or *question* you are passing to the model and include other details such as *context*, *inputs*, or *examples*. You can use these elements to instruct the model more effectively to improve the quality of results.

Let's get started by going over a basic example of a simple prompt:

Prompt

The sky is

Output:

blue.

If you are using the OpenAI Playground or any other LLM playground, you can prompt the model as shown in the following screenshot:

Playground Chat ↕

Your presets ↕ Save View code Share ...

SYSTEM
You are a helpful assistant.

USER The sky is

ASSISTANT blue.

⊕ Add message

Model
gpt-3.5-turbo ↕
Temperature 1
Maximum length 256
Stop sequences
Enter sequence and press Tab
Top P 1
Frequency penalty 0
Presence penalty 0

Here is a tutorial on how to get started with the OpenAI Playground:



Getting Started with the OpenAI Playground

Elvis Saravia



Watch on

Something to note is that when using the OpenAI chat models like `gpt-3.5-turbo` or `gpt-4`, you can structure your prompt using three different roles: `system`, `user`, and `assistant`. The system

message is not required but helps to set the overall behavior of the assistant. The example above only includes a user message which you can use to directly prompt the model. For simplicity, all of the examples, except when it's explicitly mentioned, will use only the `user` message to prompt the `gpt-3.5-turbo` model. The `assistant` message in the example above corresponds to the model response. You can also define an assistant message to pass examples of the desired behavior you want. You can learn more about working with chat models [here](#).

You can observe from the prompt example above that the language model responds with a sequence of tokens that make sense given the context `"The sky is"`. The output might be unexpected or far from the task you want to accomplish. In fact, this basic example highlights the necessity to provide more context or instructions on what specifically you want to achieve with the system. This is what prompt engineering is all about.

Let's try to improve it a bit:

Prompt:

```
Complete the sentence:  
The sky is
```

Output:

```
blue during the day and dark at night.
```

Is that better? Well, with the prompt above you are instructing the model to complete the sentence so the result looks a lot better as it follows exactly what you told it to do ("complete the sentence"). This approach of designing effective prompts to instruct the model to perform a desired task is what's referred to as **prompt engineering** in this guide.

The example above is a basic illustration of what's possible with LLMs today. Today's LLMs are able to perform all kinds of advanced tasks that range from text summarization to mathematical reasoning to code generation.

Prompt Formatting

You have tried a very simple prompt above. A standard prompt has the following format:

```
<Question>?
```

or

```
<Instruction>
```

You can format this into a question answering (QA) format, which is standard in a lot of QA datasets, as follows:

```
Q: <Question>?
```

```
A:
```

When prompting like the above, it's also referred to as *zero-shot prompting*, i.e., you are directly prompting the model for a response without any examples or demonstrations about the task you want it to achieve. Some large language models have the ability to perform zero-shot prompting but it depends on the complexity and knowledge of the task at hand and the tasks the model was trained to perform good on.

A concrete prompt example is as follows:

Prompt

```
Q: What is prompt engineering?
```

With some of the more recent models you can skip the "Q:" part as it is implied and understood by the model as a question answering task based on how the sequence is composed. In other words, the prompt could be simplified as follows:

Prompt

What is prompt engineering?

Given the standard format above, one popular and effective technique to prompting is referred to as *few-shot prompting* where you provide exemplars (i.e., demonstrations). You can format few-shot prompts as follows:

```
<Question>?
<Answer>
<Question>?
<Answer>
<Question>?
<Answer>
<Question>?
```

The QA format version would look like this:

```
Q: <Question>?
A: <Answer>
Q: <Question>?
A: <Answer>
Q: <Question>?
A: <Answer>
Q: <Question>?
A:
```

Keep in mind that it's not required to use the QA format. The prompt format depends on the task at hand. For instance, you can perform a simple classification task and give exemplars that demonstrate the task as follows:

Prompt:

```
This is awesome! // Positive
This is bad! // Negative
Wow that movie was rad! // Positive
What a horrible show! //
```

Output:

Negative

Few-shot prompts enable in-context learning, which is the ability of language models to learn tasks given a few demonstrations. We discuss zero-shot prompting and few-shot prompting more extensively in upcoming sections.

Related Learning

COURSE

Prompt Engineering for LLMs

Master zero-shot, few-shot, and advanced prompting methods to get better results from large language models.

📖 Beginner

2 hours

COURSE

Building Effective AI Agents

Learn to build effective AI agents. Covers function calling, tool integration, and debugging agentic systems.

📖 Intermediate

5 hours



Explore All Courses

Discover our full catalog of AI and prompt engineering courses. From beginners to advanced practitioners.

Use code **PROMPTING20** for 20% off!

[Browse Academy](#)

Last updated on Mon Feb 02 2026

Sponsored by **SerpApi**

Copyright © 2026 DAIR.AI